

Project Summary Report: Near-Earth Objects (NEO) Hazard Prediction

1. Introduction

For this capstone project, I chose the NASA Near-Earth Objects (NEO) dataset. The primary motivation behind selecting this data was to understand how machine learning and deep learning algorithms can be applied to astronomical and physical measurements to solve classification problems. Specifically, the goal of this project is to analyze the characteristics of various space objects (such as their size, speed, and distance from Earth) and train models that can accurately predict whether an object poses a potential danger to our planet. This represents a critical, real-world application of artificial intelligence in planetary defense and risk assessment.

2. Dataset Description

The dataset contains information regarding various tracking parameters of asteroids. It consists of multiple numerical features, including the estimated minimum and maximum diameter, the relative velocity, the miss distance from Earth, and the absolute magnitude of each object. The target label is hazardous, which indicates whether the object is a threat (True) or safe (False). During the initial exploration, the dataset was found to have a severe class imbalance. The vast majority of tracked space objects are completely harmless, while only a very small percentage are classified as hazardous. Because of this massive imbalance, standard classification accuracy would be highly misleading. Therefore, the F1-Score—which creates a balance between Precision and Recall—was chosen as the primary metric. For data preparation, non-predictive tracking features like IDs and names were removed, categorical variables were encoded into numerical values, and all numeric features were scaled using a standard scaler. The dataset was then split into 80% for training and 20% for testing using a stratified split to preserve the target class ratios.

3. Models Used

Three traditional machine learning classifiers were trained and evaluated on the dataset:

Logistic Regression: Achieved an accuracy of 90.33%, but an extremely low F1-Score of 0.1358 due to poor Recall.

Decision Tree Classifier: Achieved an accuracy of 89.11% and an F1-Score of 0.4461, detecting more hazardous objects but with more false alarms.

Random Forest Classifier: The best-performing traditional model with an accuracy of 91.75%, Precision of 0.6110, and F1-Score of 0.4980.

The Random Forest model outperformed the others because it uses multiple decision trees to vote on the final outcome, making it robust against tabular data and class imbalance.

4. Neural Network

A deep learning approach was implemented using Keras. The base network was constructed using Sequential dense layers with ReLU activation functions for the hidden layers and a Sigmoid activation function at the final output layer. To avoid overfitting, a Dropout layer was included, and an Early Stopping callback was integrated to automatically halt training at the optimal epoch. An improved network architecture with hidden layers of 64, 32, and 16 neurons and a dropout rate of 0.3 was also tested. However, the severe class imbalance made it difficult for the dense layers to identify meaningful patterns for the rare hazardous class.

5. Results & Comparison

The results showed that the Random Forest classifier remained the superior choice for this dataset.

The baseline Neural Network achieved an accuracy of 91.45% but an F1-Score of 0.2931. The improved Neural Network performed worse, with its F1-Score dropping to 0.2256 and Recall falling to 0.1329.

Traditional ensemble tree methods handle structured, tabular numerical data far more effectively than standard deep learning architectures, especially when the dataset is highly imbalanced.

6. What I Learned

This project provided several valuable lessons regarding data science workflows.

First, more complex models are not always better than traditional algorithms. For structured spreadsheet-like data, ensemble methods such as Random Forest are often faster and more accurate.

Second, checking for class imbalance is critical before building models. Accuracy alone can lead to misleading conclusions, especially in high-risk applications like planetary defense.

Finally, tuning hyperparameters and adding deeper layers can sometimes worsen performance if the underlying data structure and class ratios are not properly balanced.